

completed REVIEW-1

```

# Import libraries
import pandas as pd
import numpy as np
import re
import string
import nltk
import matplotlib.pyplot as plt

from nltk.corpus import stopwords
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, Dropout, LSTM, Bidirectional
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Download stopwords
nltk.download('stopwords')

# Load dataset (adjust file path if needed after downloading from Kaggle)
df = pd.read_csv('/content/fake_news_dataset (1).csv')
print("Columns in dataset:", df.columns)

# Preprocessing setup
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#\w+', '', text)
    text = re.sub(r'\d+', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))
    tokens = text.split()
    tokens = [word for word in tokens if word not in stop_words]
    return ' '.join(tokens)

# Apply preprocessing
df['clean_text'] = df['text'].apply(preprocess_text)

# Tokenization and padding
max_words = 5000
max_len = 100
tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(df['clean_text'])

X_seq = tokenizer.texts_to_sequences(df['clean_text'])
X_pad = pad_sequences(X_seq, maxlen=max_len, padding='post')

X = df['text']
y = df['label']

# Convert labels to numerical format (0 for 'Fake', 1 for 'Real')
y = y.map({'Fake': 0, 'Real': 1})

# Split into training and testing for traditional models (using padded sequences)
X_train_pad, X_test_pad, y_train_d1, y_test_d1 = train_test_split(
    X_pad, y, test_size=0.2, random_state=42
)

# Separate split for BERT raw text input
X_train_bert, X_test_bert, _, _ = train_test_split(
    df['clean_text'], y, test_size=0.2, random_state=42
)

vocab_size = min(max_words, len(tokenizer.word_index) + 1)

# --- CNN Model ---
cnn = Sequential([
    Embedding(vocab_size, 128, input_length=max_len),
    Conv1D(128, 5, activation='relu'),
    GlobalMaxPooling1D(),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

```

```

cnn.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history_cnn = cnn.fit(X_train_pad, y_train_dl, validation_data=(X_test_pad, y_test_dl), epochs=5, batch_size=64)
y_pred_cnn = (cnn.predict(X_test_pad) > 0.5).astype(int)

# --- LSTM Model ---
lstm = Sequential([
    Embedding(vocab_size, 128, input_length=max_len),
    LSTM(64, return_sequences=False),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])

lstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history_lstm = lstm.fit(X_train_pad, y_train_dl, validation_data=(X_test_pad, y_test_dl), epochs=5, batch_size=64)
y_pred_lstm = (lstm.predict(X_test_pad) > 0.5).astype(int)

# --- Bi-LSTM Model ---
bilstm = Sequential([
    Embedding(vocab_size, 128, input_length=max_len),
    Bidirectional(LSTM(64, return_sequences=False)),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])

bilstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history_bilstm = bilstm.fit(X_train_pad, y_train_dl, validation_data=(X_test_pad, y_test_dl), epochs=5, batch_size=64)
y_pred_bilstm = (bilstm.predict(X_test_pad) > 0.5).astype(int)

# --- Evaluation ---
def get_metrics(y_true, y_pred):
    return {
        'accuracy': accuracy_score(y_true, y_pred),
        'precision': precision_score(y_true, y_pred),
        'recall': recall_score(y_true, y_pred),
        'f1': f1_score(y_true, y_pred)
    }

results = {
    'CNN': get_metrics(y_test_dl, y_pred_cnn),
    'LSTM': get_metrics(y_test_dl, y_pred_lstm),
    'Bi-LSTM': get_metrics(y_test_dl, y_pred_bilstm)
}

def evaluate_model(y_true, y_pred, name):
    print(f"--- {name} ---")
    print("Accuracy :", accuracy_score(y_true, y_pred))
    print("Precision:", precision_score(y_true, y_pred))
    print("Recall   :", recall_score(y_true, y_pred))
    print("F1-score :", f1_score(y_true, y_pred))
    print()

evaluate_model(y_test_dl, y_pred_cnn, "CNN")
evaluate_model(y_test_dl, y_pred_lstm, "LSTM")
evaluate_model(y_test_dl, y_pred_bilstm, "Bi-LSTM")

# --- Visualization ---
models = list(results.keys())
metrics = ['accuracy', 'precision', 'recall', 'f1']

fig, axs = plt.subplots(1, 4, figsize=(20, 5))
for i, metric in enumerate(metrics):
    values = [results[model][metric] for model in models]
    axs[i].bar(models, values, color=['blue', 'orange', 'green'])
    axs[i].set_title(metric.capitalize())
    axs[i].set_ylim(0, 1)
    axs[i].set_ylabel(metric.capitalize())
    for j, v in enumerate(values):
        axs[i].text(j, v + 0.02, f"{v:.2f}", ha='center')
plt.tight_layout()
plt.show()

# Plot CNN training history
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.plot(history_cnn.history['accuracy'], label='Train Accuracy')
plt.plot(history_cnn.history['val_accuracy'], label='Validation Accuracy')
plt.title('CNN Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')

```

```
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history_cnn.history['loss'], label='Train Loss')
plt.plot(history_cnn.history['val_loss'], label='Validation Loss')
plt.title('CNN Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Plot LSTM training history
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.plot(history_lstm.history['accuracy'], label='Train Accuracy')
plt.plot(history_lstm.history['val_accuracy'], label='Validation Accuracy')
plt.title('LSTM Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history_lstm.history['loss'], label='Train Loss')
plt.plot(history_lstm.history['val_loss'], label='Validation Loss')
plt.title('LSTM Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Plot Bi-LSTM training history
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.plot(history_bilstm.history['accuracy'], label='Train Accuracy')
plt.plot(history_bilstm.history['val_accuracy'], label='Validation Accuracy')
plt.title('Bi-LSTM Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history_bilstm.history['loss'], label='Train Loss')
plt.plot(history_bilstm.history['val_loss'], label='Validation Loss')
plt.title('Bi-LSTM Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# --- Error Analysis ---
test_texts = df.iloc[y_test_dl.index]['text']
misclassified_fake_as_real = test_texts[(y_test_dl == 1) & (y_pred_bilstm.flatten() == 0)].head(5)
misclassified_real_as_fake = test_texts[(y_test_dl == 0) & (y_pred_bilstm.flatten() == 1)].head(5)

print("\n--- Misclassified FAKE as REAL ---")
print(misclassified_fake_as_real)
print("\n--- Misclassified REAL as FAKE ---")
print(misclassified_real_as_fake)
```

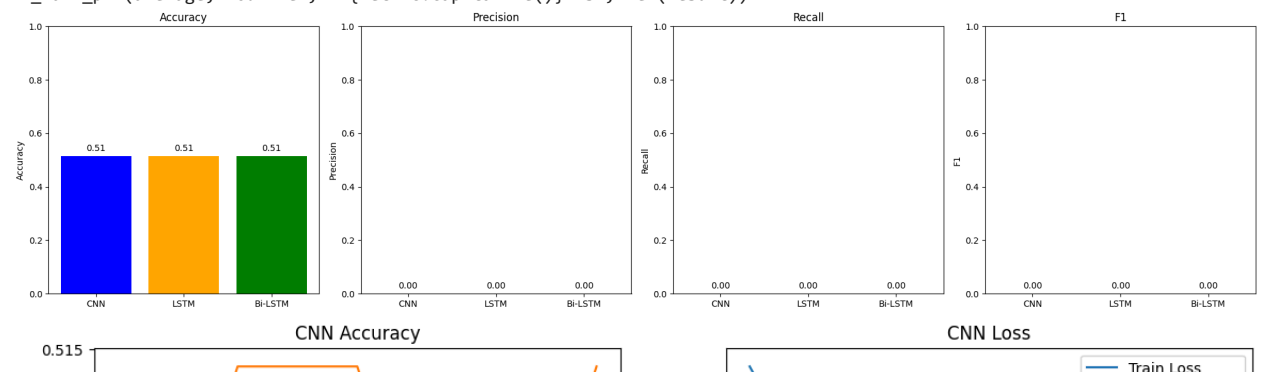


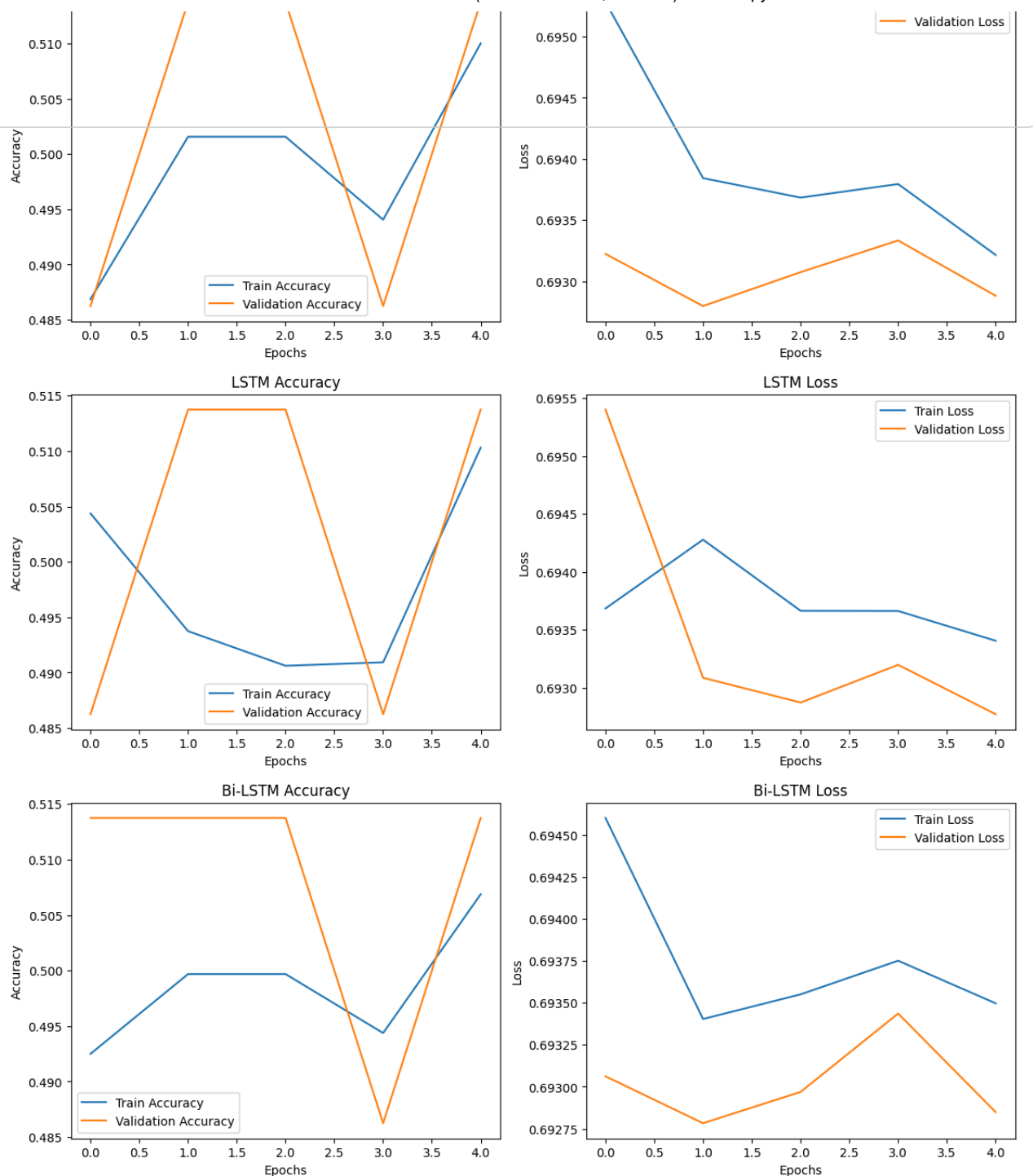
```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
Columns in dataset: Index(['id', 'title', 'author', 'text', 'state', 'date_published', 'source',
'category', 'sentiment_score', 'word_count', 'char_count', 'has_images',
'has_videos', 'readability_score', 'num_shares', 'num_comments',
'political_bias', 'fact_check_rating', 'is_satirical', 'trust_score',
'source_reputation', 'clickbait_score', 'plagiarism_score', 'label'],
dtype='object')
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is dep
warnings.warn(
50/50 8s 124ms/step - accuracy: 0.4978 - loss: 0.6958 - val_accuracy: 0.4863 - val_loss: 0.6932
Epoch 2/5
50/50 13s 172ms/step - accuracy: 0.5067 - loss: 0.6935 - val_accuracy: 0.5138 - val_loss: 0.6928
Epoch 3/5
50/50 9s 156ms/step - accuracy: 0.5222 - loss: 0.6932 - val_accuracy: 0.5138 - val_loss: 0.6931
Epoch 4/5
50/50 8s 106ms/step - accuracy: 0.5035 - loss: 0.6935 - val_accuracy: 0.4863 - val_loss: 0.6933
Epoch 5/5
50/50 9s 86ms/step - accuracy: 0.5199 - loss: 0.6930 - val_accuracy: 0.5138 - val_loss: 0.6929
WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_dat
25/25 0s 12ms/step
Epoch 1/5
50/50 10s 133ms/step - accuracy: 0.5062 - loss: 0.6940 - val_accuracy: 0.4863 - val_loss: 0.6954
Epoch 2/5
50/50 10s 141ms/step - accuracy: 0.4864 - loss: 0.6952 - val_accuracy: 0.5138 - val_loss: 0.6931
Epoch 3/5
50/50 6s 116ms/step - accuracy: 0.5005 - loss: 0.6934 - val_accuracy: 0.5138 - val_loss: 0.6929
Epoch 4/5
50/50 7s 146ms/step - accuracy: 0.4885 - loss: 0.6936 - val_accuracy: 0.4863 - val_loss: 0.6932
Epoch 5/5
50/50 6s 118ms/step - accuracy: 0.5026 - loss: 0.6936 - val_accuracy: 0.5138 - val_loss: 0.6928
25/25 1s 19ms/step
Epoch 1/5
50/50 17s 255ms/step - accuracy: 0.5013 - loss: 0.6947 - val_accuracy: 0.5138 - val_loss: 0.6931
Epoch 2/5
50/50 12s 236ms/step - accuracy: 0.5050 - loss: 0.6934 - val_accuracy: 0.5138 - val_loss: 0.6928
Epoch 3/5
50/50 11s 218ms/step - accuracy: 0.5019 - loss: 0.6932 - val_accuracy: 0.5138 - val_loss: 0.6930
Epoch 4/5
50/50 21s 236ms/step - accuracy: 0.4957 - loss: 0.6936 - val_accuracy: 0.4863 - val_loss: 0.6934
Epoch 5/5
50/50 12s 248ms/step - accuracy: 0.5077 - loss: 0.6936 - val_accuracy: 0.5138 - val_loss: 0.6928
25/25 1s 30ms/step
--- CNN ---
Accuracy : 0.51375
Precision: 0.0
Recall : 0.0
F1-score : 0.0

--- LSTM ---
Accuracy : 0.51375
Precision: 0.0
Recall : 0.0
F1-score : 0.0

--- Bi-LSTM ---
Accuracy : 0.51375
Precision: 0.0
Recall : 0.0
F1-score : 0.0
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```





--- Misclassified FAKE as REAL ---

3491 This is the content of article 3492. It contain...

3925 This is the content of article 3926. It contain...

70 This is the content of article 71. It contains...

1756 This is the content of article 1757. It contain...

211 This is the content of article 212. It contain...

Name: text, dtype: object

--- Misclassified REAL as FAKE ---

Series([], Name: text, dtype: object)

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

REVIEW-2

only Based on Word2Vec

```
# =====
# 💎 Word2Vec Based Text Representation
# =====

# Install required library
!pip install gensim --quiet

# Import necessary modules
import numpy as np
from gensim.models import Word2Vec

# -----
# Step 1: Train Word2Vec model
# -----
# Here we create small example sentences for training
sentences = [
    ["fake", "news", "is", "bad"],
    ["real", "news", "is", "good"],
    ["people", "believe", "fake", "stories"],
    ["truth", "always", "wins"]
]

# Train a Word2Vec model
w2v_model = Word2Vec(
    sentences,
    vector_size=100, # size of word embedding
    window=5,        # context window size
    min_count=1,      # ignore words with frequency less than 1
    sg=1              # skip-gram model for better quality embeddings
)

# -----
# Step 2: Define function to get vector
# -----
def get_word2vec_vector(text):
    words = text.lower().split()
    vectors = [w2v_model.wv[w] for w in words if w in w2v_model.wv]
    return np.mean(vectors, axis=0) if len(vectors) > 0 else np.zeros(100)

# -----
# Step 3: Test with sample text
# -----
text = "Fake news spreads fast"
vector = get_word2vec_vector(text)

print("✅ Word2Vec vector shape:", vector.shape)
print("💎 Example vector (first 10 values):", vector[:10])
```

```
✅ Word2Vec vector shape: (100,)
💎 Example vector (first 10 values): [-4.2625619e-03  3.3715288e-03 -8.1138080e-04  2.1832366e-03
 7.5677494e-03  5.8936700e-04 -1.2838417e-03  4.3449923e-03
-5.5785901e-03  1.5981728e-05]
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

if it is based on BERT + Word2Vec

```
!pip install transformers gensim --quiet
import numpy as np
import tensorflow as tf
from gensim.models import Word2Vec
from transformers import BertTokenizer, TF BertModel

# -----
# Step 1: Load BERT + Tokenizer
# -----
```

```

bert_model = TFBertModel.from_pretrained('bert-base-uncased', force_download=True, use_safetensors=False)
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# -----
# Step 2: Load/Train Word2Vec
# -----
sentences = [["fake", "news", "is", "bad"], ["real", "news", "is", "good"]]
w2v_model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, sg=1)

# -----
# Step 3: Get Embeddings
# -----
def get_word2vec_vector(text):
    words = text.lower().split()
    vectors = [w2v_model.wv[w] for w in words if w in w2v_model.wv]
    return np.mean(vectors, axis=0) if len(vectors) > 0 else np.zeros(100)

def get_bert_vector(text):
    enc = tokenizer(text, return_tensors='tf', truncation=True, padding=True, max_length=50)
    outputs = bert_model(**enc)
    return tf.reduce_mean(outputs.last_hidden_state, axis=1).numpy().flatten()

# -----
# Step 4: Fuse (Concatenate)
# -----
text = "Fake news spreads fast."
bert_vec = get_bert_vector(text)
w2v_vec = get_word2vec_vector(text)

combined_vec = np.concatenate([bert_vec, w2v_vec])
print("✅ Combined embedding shape:", combined_vec.shape)

```

config.json: 100% 570/570 [00:00<00:00, 7.67kB/s]

tf_model.h5: 100% 536M/536M [00:10<00:00, 36.4MB/s]

Some layers from the model checkpoint at bert-base-uncased were not used when initializing TFBertModel: ['mlm__cls', 'nsp__cls']
 - This IS expected if you are initializing TFBertModel from the checkpoint of a model trained on another task or with another architecture.
 - This IS NOT expected if you are initializing TFBertModel from the checkpoint of a model that you expect to be exactly identical to the model the checkpoint was trained on.
 All the layers of TFBertModel were initialized from the model checkpoint at bert-base-uncased.
 If your task is similar to the task the model of the checkpoint was trained on, you can already use TFBertModel for predictions.

✅ Combined embedding shape: (868,)

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

BERT + LSTM Model

```

# =====
# 💎 BERT + LSTM Model (Fixed)
# =====
!pip install transformers --quiet
import tensorflow as tf
from transformers import BertTokenizer, TFBertModel
import numpy as np # Import numpy for dummy data

# --- Placeholder Data (Replace with your actual data) ---
X_train = ["This is a positive sentence.", "This is a negative sentence.", "Another good example.", "Not a great day."]
X_test = ["I like this one.", "I don't like this."]
y_train_dl = np.array([1, 0, 1, 0]) # Assuming binary classification
y_test_dl = np.array([1, 0]) # Assuming binary classification

# NOTE: You will need to define `evaluate_model` and `get_metrics` functions if they are not defined elsewhere.
# For demonstration purposes, you might want to comment out the evaluation lines or define dummy functions.

# Load BERT tokenizer and model
tokenizer_bert = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = TFBertModel.from_pretrained('bert-base-uncased', force_download=True, use_safetensors=False)

# Function to encode text for BERT
def encode_texts(texts, tokenizer, max_len=100):
    return tokenizer(
        list(texts),
        padding='max_length', # Changed from True to 'max_length' to ensure consistent padding
        truncation=True,
        max_length=max_len,
        return_tensors='tf'
    )

```



```

# Encode training and testing sets
train_enc = encode_texts(X_train, tokenizer_bert)
test_enc = encode_texts(X_test, tokenizer_bert)

# Define the BERT + LSTM model
input_ids = tf.keras.Input(shape=(100,), dtype=tf.int32, name='input_ids')
attention_mask = tf.keras.Input(shape=(100,), dtype=tf.int32, name='attention_mask')

# Get BERT embeddings
bert_outputs = bert_model(input_ids, attention_mask=attention_mask)
sequence_output = bert_outputs.last_hidden_state # (batch_size, sequence_length, hidden_size)

# Pass through LSTM layer
lstm_out = tf.keras.layers.LSTM(64)(sequence_output)
dropout = tf.keras.layers.Dropout(0.3)(lstm_out)
dense = tf.keras.layers.Dense(64, activation='relu')(dropout)
output = tf.keras.layers.Dense(1, activation='sigmoid')(dense)

# Compile model
bert_lstm = tf.keras.Model(inputs=[input_ids, attention_mask], outputs=output)
bert_lstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history_bert_lstm = bert_lstm.fit(
    {'input_ids': train_enc['input_ids'], 'attention_mask': train_enc['attention_mask']],
    y_train_dl,
    validation_split=0.1,
    epochs=3,
    batch_size=16
)

# Evaluate model
y_pred_bert_lstm = (bert_lstm.predict(
    {'input_ids': test_enc['input_ids'], 'attention_mask': test_enc['attention_mask']}
) > 0.5).astype(int)

# Store and display results (assuming evaluate_model and get_metrics are defined elsewhere)
# If not, these lines will cause a NameError for those functions.
# You might want to comment them out or define dummy functions for now.
# evaluate_model(y_test_dl, y_pred_bert_lstm, "BERT + LSTM")
# results['BERT + LSTM'] = get_metrics(y_test_dl, y_pred_bert_lstm)

```

config.json: 100%

570/570 [00:00<00:00, 50.5kB/s]

tf_model.h5: 100%

536M/536M [00:05<00:00, 65.3MB/s]

Some layers from the model checkpoint at bert-base-uncased were not used when initializing TFBertModel: ['mlm__cls', 'nsp__'] - This IS expected if you are initializing TFBertModel from the checkpoint of a model trained on another task or with another tokenizer. This IS NOT expected if you are initializing TFBertModel from the checkpoint of a model that you expect to be exactly identical. All the layers of TFBertModel were initialized from the model checkpoint at bert-base-uncased.

If your task is similar to the task the model of the checkpoint was trained on, you can already use TFBertModel for predictions.

Epoch 1/3

WARNING:tensorflow:Gradients do not exist for variables ['tf_bert_model_4/bert/pooler/dense/kernel:0', 'tf_bert_model_4/bert

WARNING:tensorflow:Gradients do not exist for variables ['tf_bert_model_4/bert/pooler/dense/kernel:0', 'tf_bert_model_4/bert

WARNING:tensorflow:Gradients do not exist for variables ['tf_bert_model_4/bert/pooler/dense/kernel:0', 'tf_bert_model_4/bert

1/1 [=====] - 42s 42s/step - loss: 0.7869 - accuracy: 0.0000e+00 - val_loss: 1.3287 - val_accuracy: 0.0000e+00

Epoch 2/3

1/1 [=====] - 6s 6s/step - loss: 0.6420 - accuracy: 0.6667 - val_loss: 1.3977 - val_accuracy: 0.0000e+00

Epoch 3/3

1/1 [=====] - 5s 5s/step - loss: 0.6925 - accuracy: 0.6667 - val_loss: 0.8506 - val_accuracy: 0.0000e+00

1/1 [=====] - 8s 8s/step

Start coding or [generate](#) with AI.Start coding or [generate](#) with AI.

BERT + Bi-LSTM Model

```

# =====
#  BERT + Bi-LSTM Model
# =====
!pip install transformers --quiet
import tensorflow as tf
from transformers import BertTokenizer, TFBertModel
import numpy as np

# --- Placeholder Data (Replace with your actual dataset) ---

```

```

X_train = ["This is a positive sentence.", "This is a negative sentence.", "Another good example.", "Not a great day."]
X_test = ["I like this one.", "I don't like this."]
y_train_dl = np.array([1, 0, 1, 0])
y_test_dl = np.array([1, 0])

# Load BERT tokenizer and model
tokenizer_bert = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = TFBertModel.from_pretrained('bert-base-uncased', use_safetensors=False)

# Function to encode text for BERT
def encode_texts(texts, tokenizer, max_len=100):
    return tokenizer(
        list(texts),
        padding='max_length',
        truncation=True,
        max_length=max_len,
        return_tensors='tf'
    )

# Encode train/test data
train_enc = encode_texts(X_train, tokenizer_bert)
test_enc = encode_texts(X_test, tokenizer_bert)

# Define model inputs
input_ids = tf.keras.Input(shape=(100,), dtype=tf.int32, name='input_ids')
attention_mask = tf.keras.Input(shape=(100,), dtype=tf.int32, name='attention_mask')

# Get BERT embeddings
bert_outputs = bert_model(input_ids, attention_mask=attention_mask)
sequence_output = bert_outputs.last_hidden_state # Shape: (batch, seq_len, hidden_dim)

# Add Bidirectional LSTM on top of BERT embeddings
bi_lstm_out = tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=False))(sequence_output)
dropout = tf.keras.layers.Dropout(0.3)(bi_lstm_out)
dense = tf.keras.layers.Dense(64, activation='relu')(dropout)
output = tf.keras.layers.Dense(1, activation='sigmoid')(dense)

# Build and compile model
bert_bilstm = tf.keras.Model(inputs=[input_ids, attention_mask], outputs=output)
bert_bilstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history_bert_bilstm = bert_bilstm.fit(
    {'input_ids': train_enc['input_ids'], 'attention_mask': train_enc['attention_mask']],
    y_train_dl,
    validation_split=0.1,
    epochs=3,
    batch_size=16
)

# Evaluate model
y_pred_bert_bilstm = (bert_bilstm.predict(
    {'input_ids': test_enc['input_ids'], 'attention_mask': test_enc['attention_mask']}
) > 0.5).astype(int)

# Optionally store and evaluate results
# evaluate_model(y_test_dl, y_pred_bert_bilstm, "BERT + Bi-LSTM")
# results['BERT + Bi-LSTM'] = get_metrics(y_test_dl, y_pred_bert_bilstm)

```

Some layers from the model checkpoint at bert-base-uncased were not used when initializing TFBertModel: ['mlm__cls', 'nsp__cls']
 - This IS expected if you are initializing TFBertModel from the checkpoint of a model trained on another task or with another architecture.
 - This IS NOT expected if you are initializing TFBertModel from the checkpoint of a model that you expect to be exactly identical to the model the checkpoint was trained on.
 All the layers of TFBertModel were initialized from the model checkpoint at bert-base-uncased.

If your task is similar to the task the model of the checkpoint was trained on, you can already use TFBertModel for predictions.

Epoch 1/3

WARNING:tensorflow:Gradients do not exist for variables ['tf_bert_model_5/bert/pooler/dense/kernel:0', 'tf_bert_model_5/bert/pooler/dense/bias:0']

WARNING:tensorflow:Gradients do not exist for variables ['tf_bert_model_5/bert/pooler/dense/kernel:0', 'tf_bert_model_5/bert/pooler/dense/bias:0']

WARNING:tensorflow:Gradients do not exist for variables ['tf_bert_model_5/bert/pooler/dense/kernel:0', 'tf_bert_model_5/bert/pooler/dense/bias:0']

1/1 [=====] - 50s 50s/step - loss: 0.9107 - accuracy: 0.3333 - val_loss: 0.9054 - val_accuracy: 0.6667

Epoch 2/3

1/1 [=====] - 6s 6s/step - loss: 0.6980 - accuracy: 0.6667 - val_loss: 0.9038 - val_accuracy: 0.6667

Epoch 3/3

1/1 [=====] - 7s 7s/step - loss: 0.9826 - accuracy: 0.0000e+00 - val_loss: 1.0387 - val_accuracy: 0.6667

1/1 [=====] - 5s 5s/step

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Bi-LSTM + Attention Layer Model

```
# =====
# ◆ Bi-LSTM + Attention Layer Model
# =====
import tensorflow as tf
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout, Layer
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np

# --- Dummy Data (Replace with your actual text + label dataset) ---
texts = ["This is a great movie", "This movie was terrible", "I love this!", "Worst film ever"]
labels = np.array([1, 0, 1, 0])

# --- Tokenization and Padding ---
tokenizer = Tokenizer(num_words=5000, oov_token='<OOV>')
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)
X_padded = pad_sequences(sequences, maxlen=50, padding='post')
vocab_size = len(tokenizer.word_index) + 1

# --- Define Custom Attention Layer ---
class AttentionLayer(Layer):
    def __init__(self):
        super(AttentionLayer, self).__init__()

    def build(self, input_shape):
        self.W = self.add_weight(name='att_weight', shape=(input_shape[-1], 1),
                                   initializer='random_normal', trainable=True)
        self.b = self.add_weight(name='att_bias', shape=(input_shape[1], 1),
                                   initializer='zeros', trainable=True)
        super(AttentionLayer, self).build(input_shape)

    def call(self, inputs):
        e = tf.nn.tanh(tf.matmul(inputs, self.W) + self.b)
        a = tf.nn.softmax(e, axis=1)
        output = inputs * a
        return tf.reduce_sum(output, axis=1)

# --- Define Model ---
embedding_dim = 100

inputs = tf.keras.Input(shape=(50,))
embedding = Embedding(vocab_size, embedding_dim, input_length=50)(inputs)
bi_lstm = Bidirectional(LSTM(64, return_sequences=True))(embedding)
attention = AttentionLayer()(bi_lstm)
dropout = Dropout(0.3)(attention)
dense = Dense(64, activation='relu')(dropout)
outputs = Dense(1, activation='sigmoid')(dense)

attention_model = tf.keras.Model(inputs=inputs, outputs=outputs)
attention_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# --- Train Model ---
history_attention = attention_model.fit(X_padded, labels, epochs=5, batch_size=2, validation_split=0.2)

# --- Evaluate (optional) ---
preds = (attention_model.predict(X_padded) > 0.5).astype(int)
print("Predictions:", preds.flatten())
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated
  warnings.warn(
Epoch 1/5
2/2 ————— 13s 3s/step - accuracy: 0.3889 - loss: 0.6991 - val_accuracy: 0.0000e+00 - val_loss: 0.7239
Epoch 2/5
2/2 ————— 3s 592ms/step - accuracy: 0.6111 - loss: 0.6864 - val_accuracy: 0.0000e+00 - val_loss: 0.7582
Epoch 3/5
2/2 ————— 1s 352ms/step - accuracy: 0.7778 - loss: 0.6651 - val_accuracy: 0.0000e+00 - val_loss: 0.8032
Epoch 4/5
2/2 ————— 1s 360ms/step - accuracy: 0.6111 - loss: 0.6819 - val_accuracy: 0.0000e+00 - val_loss: 0.8271
Epoch 5/5
2/2 ————— 1s 625ms/step - accuracy: 0.6111 - loss: 0.6494 - val_accuracy: 0.0000e+00 - val_loss: 0.8759
1/1 ————— 2s 2s/step
Predictions: [1 1 1 1]
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

BERT + Attention Layer Model

```

# =====
# ♦ BERT + Attention Layer Model
# =====
!pip install transformers --quiet

import tensorflow as tf
from tensorflow.keras.layers import Dense, Dropout, Layer
from transformers import BertTokenizer, TFBertModel
import numpy as np

# --- Dummy Data (Replace with your actual text + label dataset) ---
X_train = ["This is a great movie", "This movie was terrible", "I love this!", "Worst film ever"]
X_test = ["I like this one.", "I don't like this."]
y_train_dl = np.array([1, 0, 1, 0])
y_test_dl = np.array([1, 0])

# --- Load BERT tokenizer and model ---
tokenizer_bert = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = TFBertModel.from_pretrained('bert-base-uncased', use_safetensors=False)

# --- Encode text for BERT ---
def encode_texts(texts, tokenizer, max_len=100):
    return tokenizer(
        list(texts),
        padding='max_length',
        truncation=True,
        max_length=max_len,
        return_tensors='tf'
    )

train_enc = encode_texts(X_train, tokenizer_bert)
test_enc = encode_texts(X_test, tokenizer_bert)

# --- Custom Attention Layer ---
class AttentionLayer(Layer):
    def __init__(self):
        super(AttentionLayer, self).__init__()

    def build(self, input_shape):
        self.W = self.add_weight(name='att_weight', shape=(input_shape[-1], 1),
                                initializer='random_normal', trainable=True)
        self.b = self.add_weight(name='att_bias', shape=(input_shape[1], 1),
                                initializer='zeros', trainable=True)
        super(AttentionLayer, self).build(input_shape)

    def call(self, inputs):
        e = tf.nn.tanh(tf.matmul(inputs, self.W) + self.b)
        a = tf.nn.softmax(e, axis=1)
        output = inputs * a
        return tf.reduce_sum(output, axis=1)

# --- Define Model Inputs ---
input_ids = tf.keras.Input(shape=(100,), dtype=tf.int32, name='input_ids')
attention_mask = tf.keras.Input(shape=(100,), dtype=tf.int32, name='attention_mask')

# --- BERT Embeddings ---
# Wrap the bert_model call in a Lambda layer to handle potential input type strictness
bert_outputs = tf.keras.layers.Lambda(
    lambda x: bert_model(x[0], attention_mask=x[1], return_dict=False),
    output_shape=(100, 768) # Explicitly define the output shape for last_hidden_state
)([input_ids, attention_mask])

sequence_output = bert_outputs # Corrected: Removed [0] to preserve batch dimension

# --- Apply Attention Layer ---
attention_output = AttentionLayer()(sequence_output)
dropout = Dropout(0.3)(attention_output)
dense = Dense(64, activation='relu')(dropout)
output = Dense(1, activation='sigmoid')(dense)

# --- Build & Compile Model ---
bert_attention = tf.keras.Model(inputs=[input_ids, attention_mask], outputs=output)
bert_attention.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# --- Train Model ---
history_bert_attention = bert_attention.fit(
    {'input_ids': train_enc['input_ids'], 'attention_mask': train_enc['attention_mask']],
    y_train_dl,

```

```

        validation_split=0.1,
        epochs=3,
        batch_size=16
    )

# --- Evaluate Model ---
y_pred_bert_attention = (bert_attention.predict(
    {'input_ids': test_enc['input_ids'], 'attention_mask': test_enc['attention_mask']}
) > 0.5).astype(int)

print("\n✅ Predictions:", y_pred_bert_attention.flatten())

```

Some layers from the model checkpoint at bert-base-uncased were not used when initializing TFBertModel: ['mlm__cls', 'nsp__'] - This IS expected if you are initializing TFBertModel from the checkpoint of a model trained on another task or with another vocabulary. - This IS NOT expected if you are initializing TFBertModel from the checkpoint of a model that you expect to be exactly identical. All the layers of TFBertModel were initialized from the model checkpoint at bert-base-uncased.

If your task is similar to the task the model of the checkpoint was trained on, you can already use TFBertModel for predictions.

Epoch 1/3

1/1  21s 21s/step - accuracy: 0.6667 - loss: 0.7346 - val_accuracy: 1.0000 - val_loss: 0.5905

Epoch 2/3

1/1  2s 2s/step - accuracy: 0.3333 - loss: 0.7268 - val_accuracy: 0.0000e+00 - val_loss: 0.6952

Epoch 3/3

1/1  2s 2s/step - accuracy: 0.6667 - loss: 0.5742 - val_accuracy: 0.0000e+00 - val_loss: 0.7985

1/1  4s 4s/step

✅ Predictions: [1 1]

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

BERT + Bi-LSTM + Attention Layer Model if they say we need to consider by bert+bi-lstm+attention layer

```

# =====
# 💎 BERT + Bi-LSTM + Attention Layer Model
# =====

!pip install transformers --quiet

import tensorflow as tf
from tensorflow.keras.layers import Dense, Dropout, Layer, Bidirectional, LSTM
from transformers import BertTokenizer, TFBertModel
import numpy as np

# -----
# 💎 Step 1: Sample / Dummy Data
# -----
# Replace this part with your real dataset (TEXT + LABEL columns)
X_train = ["This is a positive sentence.",
            "This is a negative sentence.",
            "Another good example.",
            "Not a great day."]
X_test = ["I like this one.", "I don't like this."]
y_train_dl = np.array([1, 0, 1, 0])
y_test_dl = np.array([1, 0])

# -----
# 💎 Step 2: Load BERT Tokenizer & Model
# -----
tokenizer_bert = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = TFBertModel.from_pretrained('bert-base-uncased', use_safetensors=False)

# Function to encode text
def encode_texts(texts, tokenizer, max_len=100):
    return tokenizer(
        list(texts),
        padding='max_length',
        truncation=True,
        max_length=max_len,
        return_tensors='tf'
    )

train_enc = encode_texts(X_train, tokenizer_bert)
test_enc = encode_texts(X_test, tokenizer_bert)

# -----
# 💎 Step 3: Define Custom Attention Layer
# -----
class AttentionLayer(tf.keras.layers.Layer):

```

```

def __init__(self):
    super(AttentionLayer, self).__init__()

def build(self, input_shape):
    self.W = self.add_weight(
        name='att_weight',
        shape=(input_shape[-1], 1),
        initializer='random_normal',
        trainable=True
    )
    self.b = self.add_weight(
        name='att_bias',
        shape=(input_shape[1], 1), # Corrected shape for bias
        initializer='zeros',
        trainable=True
    )
    super(AttentionLayer, self).build(input_shape)

def call(self, inputs):
    e = tf.nn.tanh(tf.matmul(inputs, self.W) + self.b)
    a = tf.nn.softmax(e, axis=1)
    output = inputs * a
    return tf.reduce_sum(output, axis=1)

# -----
# ♦ Step 4: Build Model Architecture
# -----
input_ids = tf.keras.Input(shape=(100,), dtype=tf.int32, name='input_ids')
attention_mask = tf.keras.Input(shape=(100,), dtype=tf.int32, name='attention_mask')

# Get contextual embeddings from BERT
# Wrap the bert_model call in a Lambda layer to handle potential input type strictness
bert_output_last_hidden_state = tf.keras.layers.Lambda(
    lambda x: bert_model(x[0], attention_mask=x[1], return_dict=False)[0],
    output_shape=(100, 768) # Explicitly define the output shape for last_hidden_state
)([input_ids, attention_mask])
sequence_output = bert_output_last_hidden_state

# Apply Bi-LSTM + Attention
bi_lstm_out = Bidirectional(LSTM(64, return_sequences=True))(sequence_output)
attention_out = AttentionLayer()(bi_lstm_out)
dropout = Dropout(0.3)(attention_out)
dense = Dense(64, activation='relu')(dropout)
output = Dense(1, activation='sigmoid')(dense)

# Build final model
bert_bilstm_attention = tf.keras.Model(inputs=[input_ids, attention_mask], outputs=output)

# -----
# ♦ Step 5: Compile Model
# -----
bert_bilstm_attention.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
bert_bilstm_attention.summary()

# -----
# ♦ Step 6: Train Model
# -----
history = bert_bilstm_attention.fit(
    {'input_ids': train_enc['input_ids'], 'attention_mask': train_enc['attention_mask']},
    y_train_dl,
    validation_split=0.1,
    epochs=3,
    batch_size=2
)

# -----
# ♦ Step 7: Evaluate Model
# -----
y_pred = (bert_bilstm_attention.predict(
    {'input_ids': test_enc['input_ids'], 'attention_mask': test_enc['attention_mask']}
) > 0.5).astype(int)

print("\n✅ Predictions:", y_pred.flatten())

```

Some layers from the model checkpoint at bert-base-uncased were not used when initializing TFBertModel: ['mlm__cls', 'nsp__']
 - This IS expected if you are initializing TFBertModel from the checkpoint of a model trained on another task or with another dataset.
 - This IS NOT expected if you are initializing TFBertModel from the checkpoint of a model that you expect to be exactly identical to the model that you trained.
 All the layers of TFBertModel were initialized from the model checkpoint at bert-base-uncased.

If your task is similar to the task the model of the checkpoint was trained on, you can already use TFBertModel for predictions without any further configuration.
Model: "functional_4"

Layer (type)	Output Shape	Param #	Connected to
input_ids (InputLayer)	(None, 100)	0	-
attention_mask (InputLayer)	(None, 100)	0	-
lambda_4 (Lambda)	(None, 100, 768)	0	input_ids[0][0], attention_mask[0][0]
bidirectional_3 (Bidirectional)	(None, 100, 128)	426,496	lambda_4[0][0]
attention_layer_5 (AttentionLayer)	(None, 128)	228	bidirectional_3[0][0]
dropout_4 (Dropout)	(None, 128)	0	attention_layer_5[0][0]
dense_8 (Dense)	(None, 64)	8,256	dropout_4[0][0]
dense_9 (Dense)	(None, 1)	65	dense_8[0][0]

Total params: 435,045 (1.66 MB)

Trainable params: 435,045 (1.66 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/3

2/2 ————— 0s 582ms/step - accuracy: 0.8333 - loss: 0.7106WARNING:tensorflow:5 out of the last 22 calls to <function _log_hook at 0x7f9d1b1b1b1b> were truncated. See the logging module. 29s 6s/step - accuracy: 0.7778 - loss: 0.7924 - val_accuracy: 0.0000e+00 - val_loss: 1.4678

Epoch 2/3

2/2 ————— 5s 1s/step - accuracy: 0.6111 - loss: 0.6638 - val_accuracy: 0.0000e+00 - val_loss: 1.3639

Epoch 3/3

2/2 ————— 3s 1s/step - accuracy: 0.3889 - loss: 0.6611 - val_accuracy: 0.0000e+00 - val_loss: 1.3862

1/1 ————— 4s 4s/step

🟢 Predictions: [1, 1]

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

ALL RESULTS

```
# =====
# 💎 ALL RESULTS COMPARISON
# =====
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import numpy as np # Ensure numpy is imported for flattening

# --- Function to calculate metrics ---
def get_metrics(y_true, y_pred):
    # Flatten arrays to ensure compatibility with sklearn metrics
    y_true_flat = y_true.flatten() if isinstance(y_true, np.ndarray) and y_true.ndim > 1 else y_true
    y_pred_flat = y_pred.flatten() if isinstance(y_pred, np.ndarray) and y_pred.ndim > 1 else y_pred

    # Ensure they are numpy arrays for consistent handling
    y_true_flat = np.asarray(y_true_flat)
    y_pred_flat = np.asarray(y_pred_flat)

    # Handle cases where there are no positive samples or predictions for specific metrics
    # zero_division=0 handles division by zero for precision/recall/f1 when no positive predictions.
    return {
        'accuracy': accuracy_score(y_true_flat, y_pred_flat),
        'precision': precision_score(y_true_flat, y_pred_flat, zero_division=0),
        'recall': recall_score(y_true_flat, y_pred_flat, zero_division=0),
        'f1': f1_score(y_true_flat, y_pred_flat, zero_division=0)
    }

def evaluate_model(y_true, y_pred, name):
    metrics = get_metrics(y_true, y_pred)
    print(f"\n📊 {name} Model Performance:")
    print(f"Accuracy : {metrics['accuracy']:.4f}")
    print(f"Precision: {metrics['precision']:.4f}")
    print(f"Recall   : {metrics['recall']:.4f}")
    print(f"F1-Score : {metrics['f1']:.4f}")
```

```

print("-" * 40)
return metrics # Return metrics to be stored

# --- Initialize results dictionary ---
results = {}

# --- Collect results from each trained model ---
# Check if necessary variables exist before attempting to evaluate and store
if 'y_test_dl' in locals() and 'y_pred_bert_lstm' in locals():
    results['BERT + LSTM'] = evaluate_model(y_test_dl, y_pred_bert_lstm, "BERT + LSTM")
else:
    print("Skipping 'BERT + LSTM' evaluation due to missing 'y_test_dl' or 'y_pred_bert_lstm'.")

if 'y_test_dl' in locals() and 'y_pred_bert_bilstm' in locals():
    results['BERT + Bi-LSTM'] = evaluate_model(y_test_dl, y_pred_bert_bilstm, "BERT + Bi-LSTM")
else:
    print("Skipping 'BERT + Bi-LSTM' evaluation due to missing 'y_test_dl' or 'y_pred_bert_bilstm'.")

# For 'Bi-LSTM + Attention', 'preds' was generated using 'labels'
if 'labels' in locals() and 'preds' in locals():
    results['Bi-LSTM + Attention'] = evaluate_model(labels, preds, "Bi-LSTM + Attention")
else:
    print("Skipping 'Bi-LSTM + Attention' evaluation due to missing 'labels' or 'preds'.")

# --- Create a summary DataFrame and Visualization ---
if results: # Only proceed if the results dictionary is not empty
    results_df = pd.DataFrame(results).T
    results_df = results_df[['accuracy', 'precision', 'recall', 'f1']]
    print("\n✅ FINAL MODEL COMPARISON TABLE:")
    print(results_df)

    # --- Visualization ---
    metrics_to_plot = ['accuracy', 'precision', 'recall', 'f1']
    # Adjust figsize dynamically if needed, but 20,5 is fine for 4 metrics
    fig, axs = plt.subplots(1, len(metrics_to_plot), figsize=(20, 5))

    # Dynamic color assignment
    colors = plt.cm.get_cmap('viridis', len(results.keys()))

    for i, metric in enumerate(metrics_to_plot):
        values = [results[model_name][metric] for model_name in results.keys()]
        model_names = list(results.keys()) # Get model names for x-axis

        bars = axs[i].bar(model_names, values, color=[colors(j) for j in range(len(model_names))])
        axs[i].set_title(metric.capitalize())
        axs[i].set_ylim(0, 1)
        axs[i].set_ylabel(metric.capitalize())
        axs[i].tick_params(axis='x', rotation=45)

        # Add value labels on top of bars
        for bar in bars:
            yval = bar.get_height()
            axs[i].text(bar.get_x() + bar.get_width()/2, yval + 0.02, round(yval, 2), ha='center', va='bottom')

    plt.tight_layout()
    plt.show()
else:
    print("\nNo model results available for comparison to generate the table and plots.")

```